[BUSINESS] FORECASTING AND PREDICTIVE MODELING

LECTURE 12

Lect. PhD. Oneț-Marian Zsuzsanna

Babeș - Bolyai University
Computer Science and Mathematics Faculty

2025 - 2026

In Lecture 11 ...

- Dynamic regression models

Today

- Prophet
- Neural network-based models
- The written exam

Generalized Additive Model

- Business-related time series usually have a trend and one or more seasonal components. Also, they tend to be influenced by holidays.
- A well-recognized way to model such a time series is to use the **Generalized Additive Model (GAM)**.
  - A time series is composed of several components and each has a linear of non-linear relation to the time series
  - These components can be estimated in parallel and then added up.

Prophet I

- Prophet is a model developed by researchers at Facebook. It was described in detail in a 2018 the paper: *S. J. Taylor, B. Letham: Forecasting at Scale, The American Statistician, 72(1), 37–45.*

- The basic idea of Prophet is to describe a time series as:

$$y_t = g(t) + s(t) + h(t) + \epsilon_t$$

- where:
  - g(t) - is the trend component
  - s(t) - is describes different seasonal patterns
  - h(t) - captures the holiday effects
  - ϵ_t is the error term (expected to be white noise)

Prophet II

- For the trend component, Prophet uses either a piecewise linear model (with automatic changepoint detection) or a saturating growth model.
- For the seasonal component Fourier terms are used
- Holidays are modeled as dummy variables.

Example - Python I

- Let us consider the number of visitors to an academic teaching notes website.

```
visitors = pd.read_csv("daily-website-visitors.csv", index_col = 'Row',
    thousands=",")

visitors["Date"] = pd.to_datetime(visitors["Date"])
visitors["First.Time.Visits"] = pd.to_numeric(visitors["First.Time.Visits"],
    errors = 'coerce')
#keep only the data and the first time visitors
visitors = visitors[['Date', 'First.Time.Visits']]

visitors = visitors.rename(columns = {'Date': 'ds', 'First.Time.Visits': 'y'})

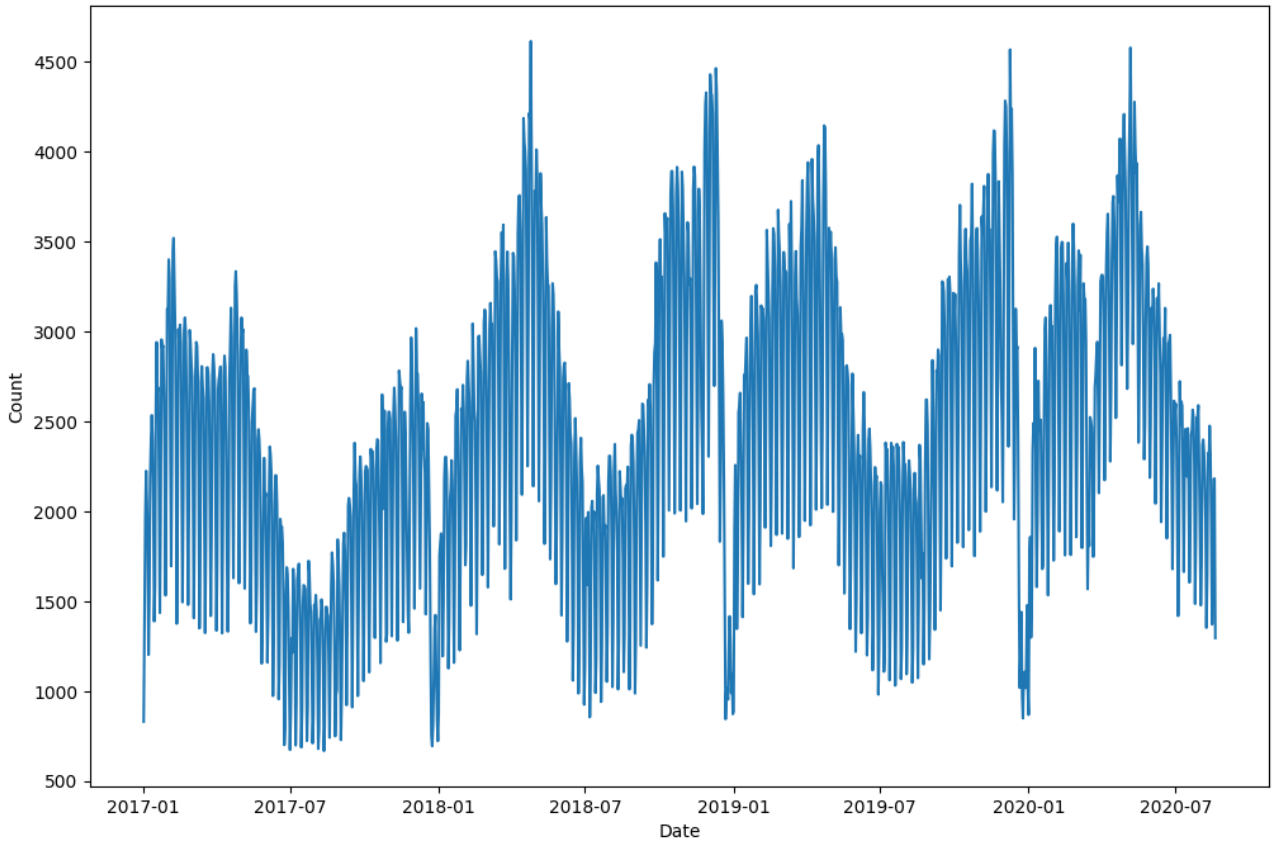
#filter out only data from 01.01.2017
visitors = visitors[visitors['ds'] >= pd.to_datetime('2017-01-01')]
```

Example - Python II

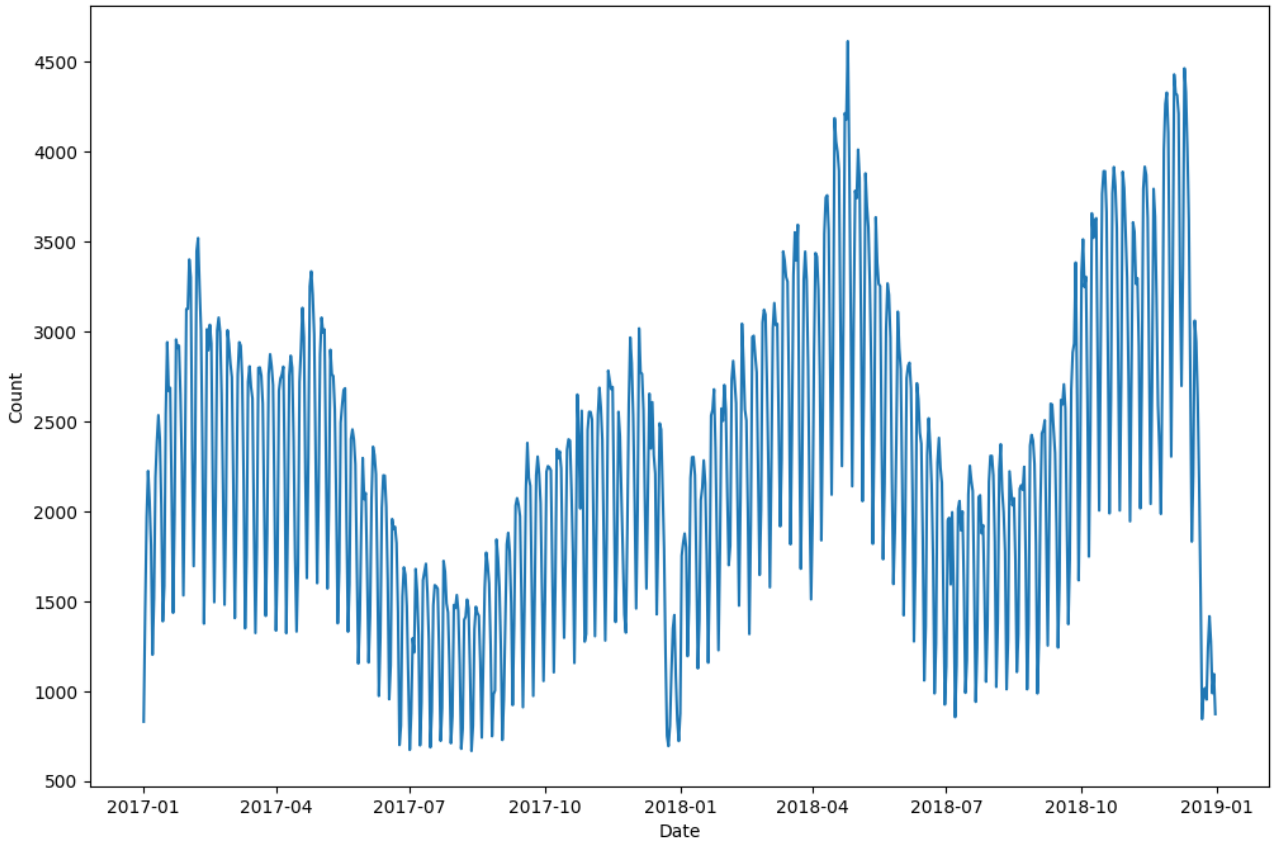
```
import matplotlib.pyplot as plt

plt.figure(figsize = (12,8))
plt.plot(visitors['ds'], visitors['y'])
plt.xlabel("Date")
plt.ylabel("Count")
plt.show()
```

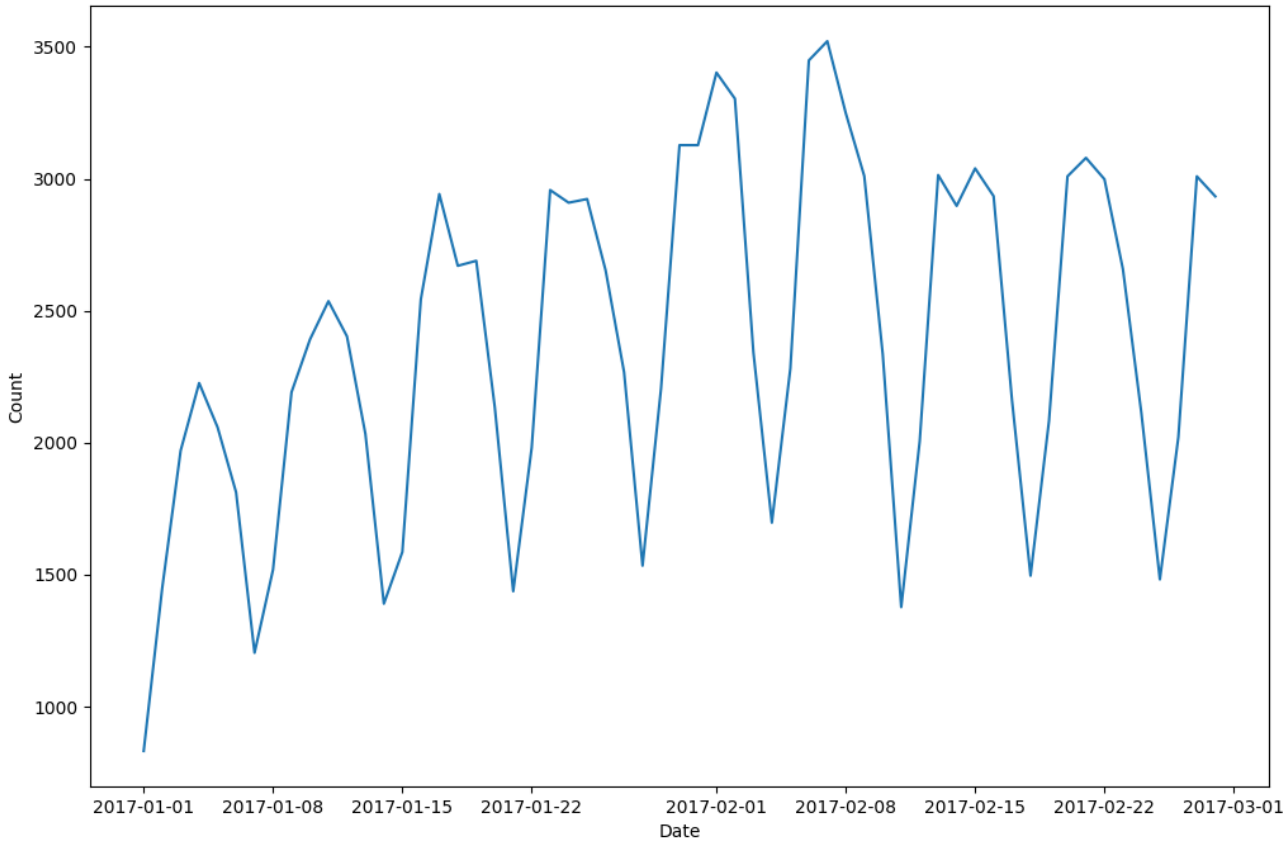
Example - Python III



Example - Python IV



Example - Python V



Example - Python VI

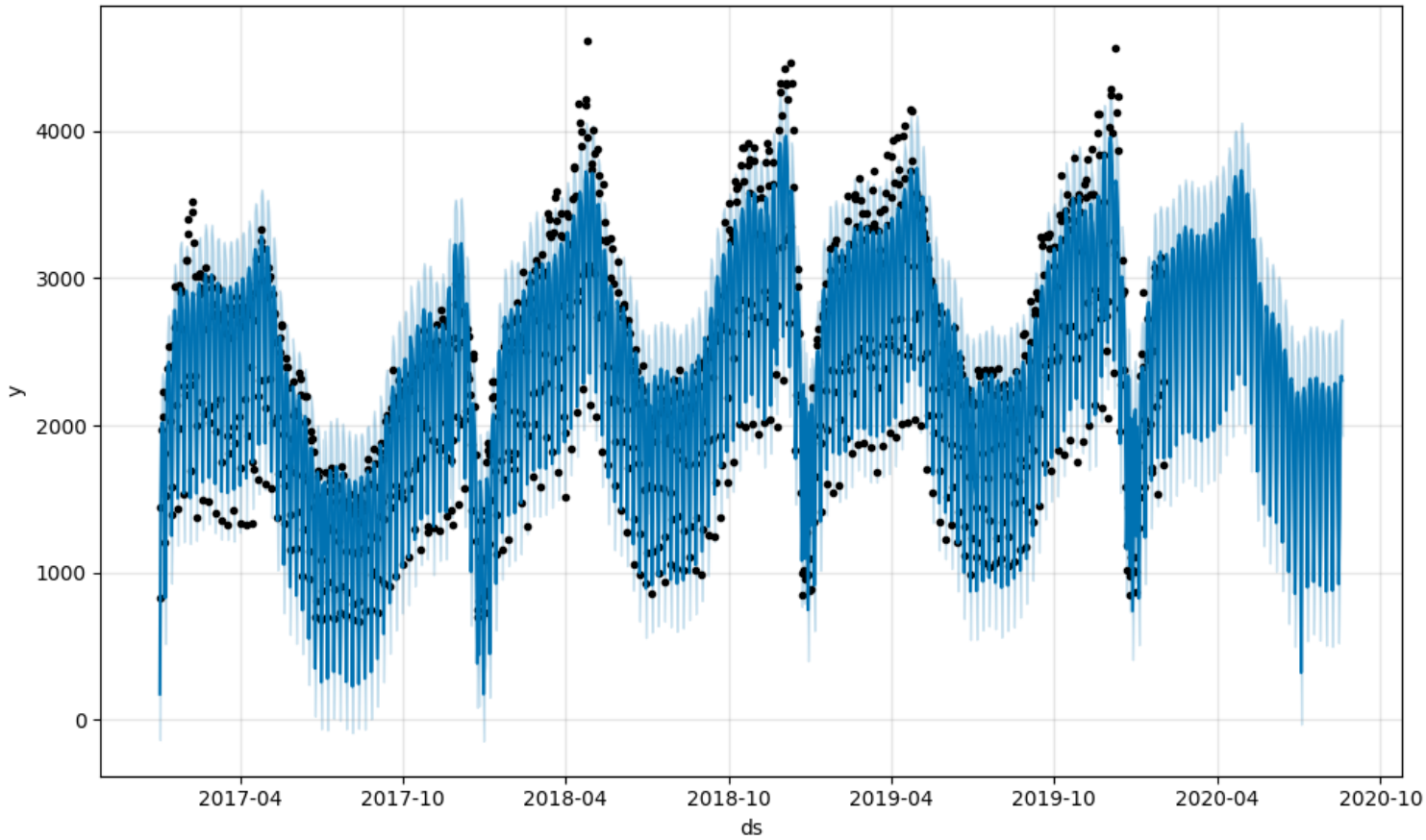
```
#build the prophet model

from prophet import Prophet
model = Prophet()
model.add_country_holidays(country_name='US') #add the dummies for the holidays
model.fit(train)

future = model.make_future_dataframe(periods = len(test), freq = 'd')
print(future.tail())
forecast = model.predict(future)
forecast.tail()

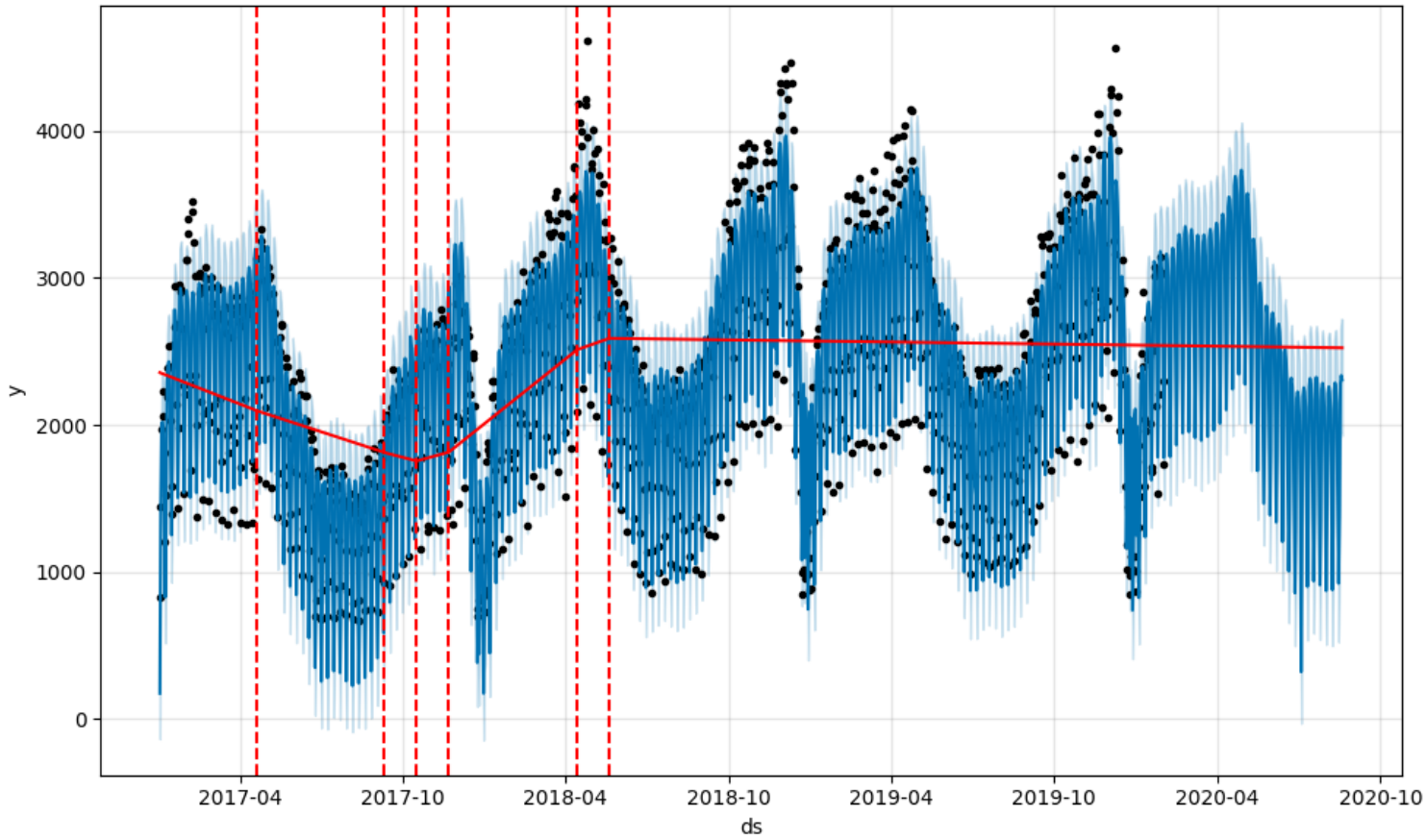
fig = model.plot(forecast)
```

Example - Python VII



Example - Python VIII

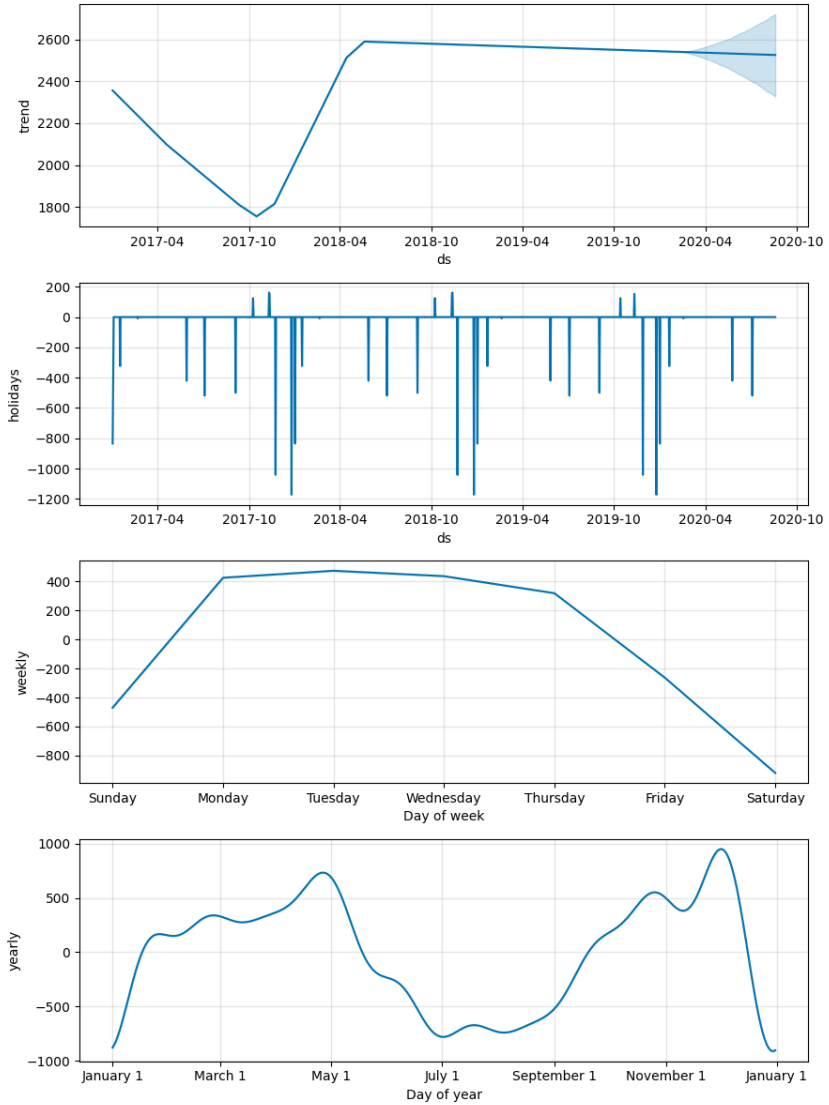
```
a = add_changepoints_to_plot(fig.gca(), model, forecast)
```



Example - Python IX

```
model.plot_components(forecast)
```

Example - Python X



Example - Python XI

```
from sklearn import metrics
import numpy as np

prediction = forecast.loc[train_size:, 'yhat']

mape = metrics.mean_absolute_percentage_error(test['y'], prediction)
mae = metrics.mean_absolute_error(test['y'], prediction)
mse = metrics.mean_squared_error(test['y'], prediction)

print("Mape: ", mape)
print("Mae: ", mae)
print("Mse: ", mse)
```

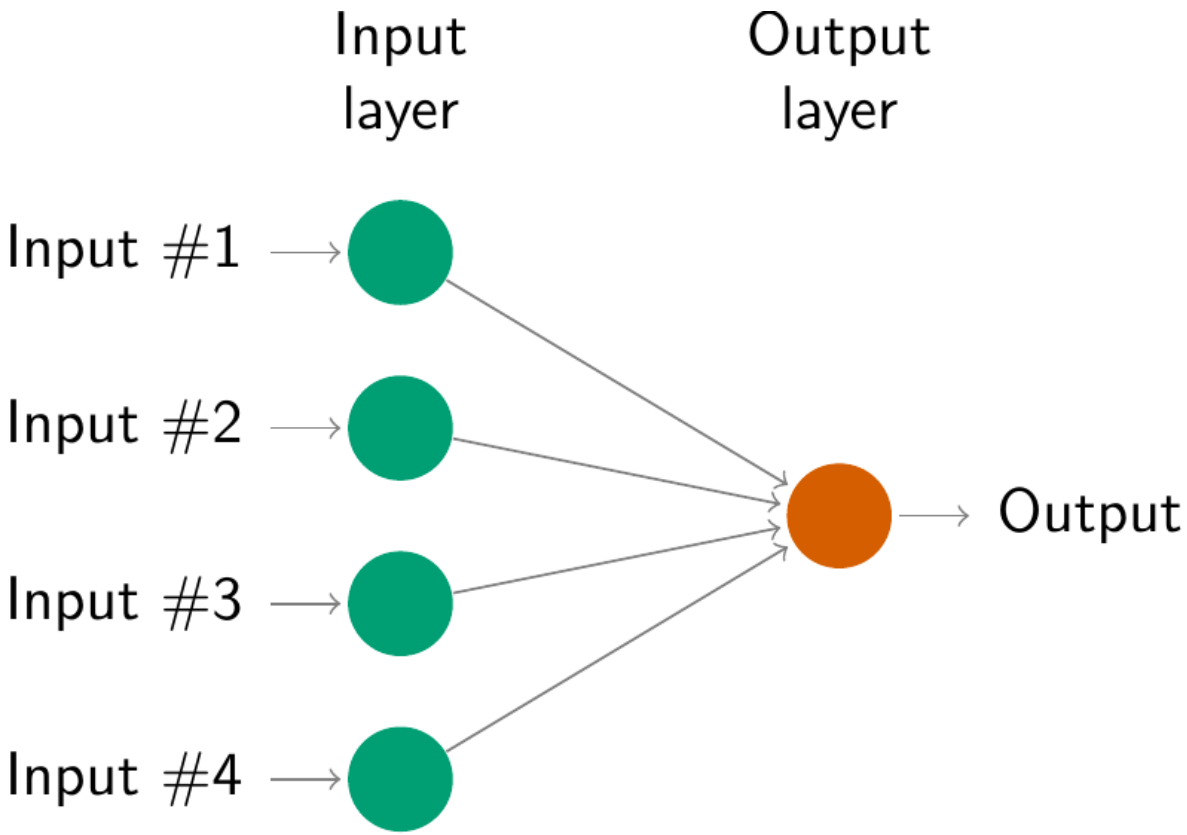
Example - Python XII

```
from prophet.diagnostics import cross_validation
model_cv = cross_validation(model,
initial='100 days',
period='180 days',
horizon = '365 days')
print(model_cv)
```

Neural network-based models I

- An artificial neural network is a network made of neurons, organized in layers. Nodes from one layer are connected to nodes from the next layer. The simplest ANN architecture contains one single input layer and one output layer of neurons.

Neural network-based models II

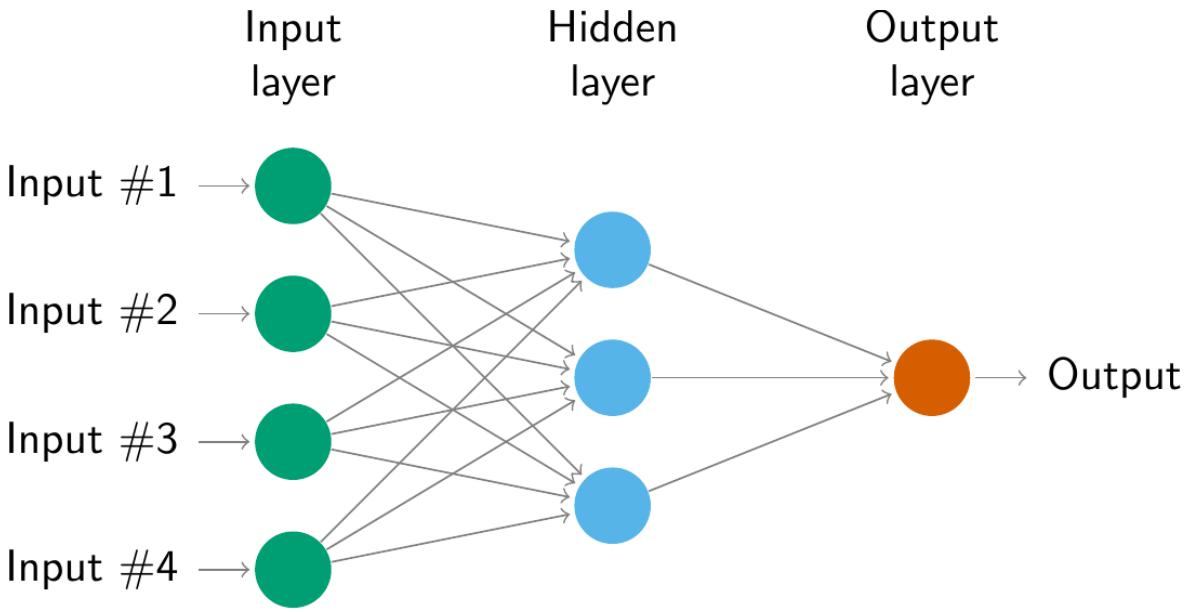


Source: https://otexts.com/fpp3/nnetar.html

Neural network-based models III

- Such a simple architecture is very similar to linear regression.
- The input nodes are the predictors and each has an associated weight, plus we have a bias term as well.
- The goal of the model is to learn the optimal weights (such that a cost function, for example MSE, is minimized)
- We can have more complex architectures, by adding hidden layers

Neural network-based models IV



Source: https://otexts.com/fpp3/nnetar.html

Neural network-based models V

- In the above architecture we have one hidden layer. The input for a node j from the hidden layer is:

$$z_j = b_j + \sum_{i=1}^{4} w_{i,j} x_i$$

- This value is in general passed through a non-linear function (called activation function), before becoming the output of the node. One popular example for the activation function is the sigmoid function:

$$s_z = \frac{1}{1 + e^{-z}}$$

Neural network-based models VI

- In most cases, ANNs are used for *tabular data*, we have a set of attributes as input and a predictor variable which will be the output of the model.
- In case of time series data, we can use as input lagged values of the series. This is called a neural network autoregressor model, or NNAR.
- Considering an architecture with one single hidden layer, a NNAR model architecture can be described by two values: the number of nodes in the input layer (i.e. how many lagged values to use) and the number of nodes in the hidden layer.
- For example, NNAR(9,5) means an ANN model with 9 input nodes and 5 hidden nodes (and obviously, we have one output node as well in the architecture, but that is default, it is not specified separately in the model).

Neural network-based models VII

- In case of seasonal data, it might be useful to consider the last observed value(s) from the same season as input as well.
- In such cases, NNAR has 3 parameters, for example: $NNAR(3,1,2)_{12}$, means monthly data with yearly seasonality, where for the forecast of y_t we use as predictor variables $y_{t-1}, y_{t-2}, y_{t-3}$ and y_{t-12}. Also, the hidden layer has 2 nodes.